

Hand Motion Reconstruction from Minimal Inputs through Latent-Space Learning

Charlotte Ludovica Primiceri*, Serena Trovalusci*, Diana Ioana Bubenek Turconi*, Giordano Pagano*, Emanuele De Santis, Marilena Vendittelli

Dipartimento di Ingegneria Informatica, Automatica e Gestionale (DIAG), Sapienza University of Rome

*These authors contributed equally to this work.

Abstract—We present a real-time, data-driven framework for hand motion retargeting that reconstructs full joint configurations of a virtual hand from only four high-level inputs (three closures and one abduction) acquired from a Weart TouchDIVER G1 glove (Fig. 1). We investigate Fully-Connected and Transformer-based models and enforce biomechanical plausibility with an encoder-based latent loss learned via an autoencoder. The approach yields anatomically consistent poses with low latency in Unity, despite minimal sensing.

I. INTRODUCTION

This project addresses the problem of real-time motion retargeting for a human hand model by leveraging a data-driven approach based on a neural network, as an alternative to classical direct or inverse kinematics techniques. The network is trained to predict the complete joint configuration of a virtual hand from a reduced set of input features. Specifically, the available inputs consist of the closure parameters of the thumb, index, and middle fingers, as well as the abduction parameter of the thumb. These signals are acquired in real time from the Weart TouchDIVER G1 haptic glove (Fig. 1), which provides continuous hand motion data. The objective is to reconstruct a plausible and consistent full-hand pose in a virtual environment using only these partial measurements.

Existing retargeting pipelines often rely on dense sensing (e.g., full keypoint tracking) or optimization-heavy kinematics, which limits portability and latency in interactive settings. We instead pursue minimal sensing (four scalars) with a learning-based model that restores a full, biomechanically consistent pose.

Contributions. We (i) formulate minimal-input retargeting from four scalars (three closures + one abduction) to a full 45-DoF hand, (ii) design a Transformer predictor with per-finger output heads, (iii) enforce hand pose plausibility via an encoder-based latent loss, achieving a mean absolute error of 10° with a 15-dimensional latent representation, (iv) handle angle periodicity by predicting sine/cosine pairs, and (v) deliver a real-time Unity pipeline (20 Hz acquisition).

The Weart TouchDIVER G1 is a lightweight, wearable haptic device that integrates lifelike force feedback, texture



Fig. 1: TouchDIVER G1

rendering, and thermal cues. It is equipped with precise sensors for tracking hand and finger movements, allowing for natural and intuitive interaction with virtual environments and objects [1].

If used directly through the Weart SDK in Unity, the glove already provides a virtual hand model that animates smoothly. However, this model functions as a black box: it interpolates the glove's sensor inputs through internal assumptions that are not accessible to the user. While the resulting motion appears realistic, it does not provide control over the underlying joint angles and therefore limits its applicability in research and advanced retargeting tasks.

The motion retargeting system was developed and tested within a Unity-based simulated environment, using the official Weart SDK and documentation available on the Weart website. This setup enabled the integration of real-time data acquisition, neural network inference, and visual feedback of the virtual hand motion [2].

We developed a complete pipeline consisting of:

- A custom C# logging tool that records synchronized joint rotation angles and sensor inputs, used to build the training dataset.
- Two machine learning models—a Fully Connected Neural Network (FCNN) and a Transformer-based model—trained to map a 4-dimensional input vector to 62 outputs representing processed joint angle information.

- A client-server architecture in which Unity sends real-time sensor data to a Python-based server, which returns the predicted joint angles to drive the animation of the virtual hand in real time.

Human hand movements exhibit synergies, meaning that many joints move in coordinated and highly correlated patterns rather than independently. These natural correlations between finger joints imply that it is not necessary to represent or predict every single joint angle to reconstruct realistic hand motions. Instead, a reduced set of parameters can sufficiently capture the essential variability of hand poses. This topic has been a subject of research in the study by Santello, Flanders and Soechting [3], and their approach will be applied in the present work.

To exploit this property and improve both generalization and efficiency, we apply dimensionality reduction techniques such as Principal Component Analysis (PCA) and Autoencoders. These methods allow us to represent hand configurations in a compact latent space, enabling the network to generate anatomically plausible hand motions from a limited set of input signals, with low latency.

II. RELATED WORK

Motion retargeting for the human hand has been explored through several distinct paradigms, including direct mapping, optimization, and data-driven methods.

Direct and Kinematic Mapping. Early approaches often relied on joint-space mapping, where each human joint is directly associated with a robot joint through predefined rules. While intuitive, this strategy fails to capture anatomical and kinematic differences, leading to unnatural poses. Other methods adopted Cartesian mapping, linking human keypoints to robot end-effectors and solving inverse kinematics (IK) to recover robot joint values. These methods, however, can be brittle and computationally demanding.

Optimization-Based Methods. Another line of research formulates motion retargeting as a constrained optimization problem. ByteDexter [4] is a representative example, where a 20-DoF robotic hand is controlled by minimizing discrepancies between human and robot keyvectors under kinematic and collision constraints. This yields high-fidelity teleoperation but at a significant computational cost, making it less suitable for applications where low latency is critical.

Data-Driven and Neural Methods. More recently, neural networks have become the dominant approach. Methods like GeoRT [5] introduced constrained loss formulations with explicit geometric objectives (e.g., collision avoidance, motion preservation), achieving high-frequency teleoperation. A common limitation of such work, however, is the reliance on dense sensor inputs, such as tracking all fingertips, without explicitly leveraging dimensionality reduction to model the natural, synergistic correlations in hand motion. These dense inputs can be acquired from

motion capture systems or vision-based estimators. In this context, Zimmermann and Brox [6] demonstrated that convolutional neural networks trained on motion capture annotated datasets can directly regress 3D hand poses from monocular RGB input, laying the foundation for many subsequent vision-based methods. Nevertheless, they still rely on external cameras and are vulnerable to occlusions and lighting, which limits their practicality in real-time, portable applications. In parallel, glove-based data-driven methods are carried on, such as the work of Glauser et al. [7] which reconstructs continuous finger motion from sparse stretch sensors embedded in the glove using a regression model that maps sensor readings directly to joint angles. However, this approach still requires multiple sensors and user-specific calibration, which can limit its scalability.

Transformers for Motion Synthesis. Our work also draws from advancements in sequence modeling for motion generation. Originally designed for natural language processing [8], the Transformer architecture has proven highly effective for modeling sequential data like human motion. Its self-attention mechanism excels at capturing the complex, long-range dependencies between different elements in a sequence. Consequently, Transformers have been successfully applied to tasks like full-body motion prediction and gesture synthesis, where they generate more coherent and natural movements than prior recurrent models.

Latent-space priors and encoder-based losses. A complementary line of research learns low-dimensional manifolds of plausible hand configurations and supervises predictors in that space rather than on raw joint angles. Guo et al. show that pairing Transformer backbones with a generative latent bottleneck enables structured motion editing by optimizing objectives directly in the latent space [9], while Portnova-Fahreva et al. use an autoencoder over myoelectric signals and enforce control in the encoder’s coordinates to regularize predictions toward feasible prosthetic hand poses [10]. By training our Transformer to minimize error within this latent space, we impose a powerful data-driven constraint. This ensures the generated outputs lie on the manifold of natural motion, effectively capturing the complex, nonlinear synergies of the human hand. Compared with PCA-in-loss formulations, a nonlinear autoencoder better captures inter-joint dependencies, providing a stronger plausibility prior while remaining compatible with real-time inference in our pipeline.

III. METHODOLOGY

The goal of this work is to learn a mapping from glove-derived parameters to a complete 3D pose of a human hand. The input consists of four scalar parameters: three closure values (thumb, index, middle), and one abduction value for the thumb. The output targets are the rotational configurations of 15 joints (three per finger),

each represented along three rotational axes (XYZ), for a total of 45 joint angles.

To address issues of discontinuity inherent in Euler angle representation, several rotational components were encoded using their sine and cosine values. This increased the output dimensionality to 62, allowing the network to represent circular quantities more robustly.

A. Data acquisition

Data collection was carried out in three phases. We initially recorded around 3,000 samples from the thumb, index, and middle fingers, which provided a limited yet sufficient starting point for early experiments. We started with three closure parameters. Successively, since thumb movements are more complex and have a wider range, we decided to consider a more accurate description by including the abduction parameter of the thumb. We followed with an expanded dataset of approximately 11,000 samples, still restricted to the same three fingers. Finally, we acquired a full dataset of 17,000 samples that included all five fingers.

Each data sample in the final dataset consists of:

- 45 joint angle values (3 rotational axes $\times$ 15 joints)
- 4 input parameters (ThumbClosure, IndexClosure, MiddleClosure, ThumbAbduction)
- 1 timestamp indicating when the sample was recorded

All angles were recorded in Euler format with respect to the palm's coordinate frame, and all signals were synchronized and stored in CSV format using a custom Unity script.

Early Post-Processing Attempts: In the first stages, we encountered unrealistic flipping and jumps in the predicted joint motions, especially in some specific joints:

- All joints of the thumb (Thumb01, Thumb02, Thumb03), across all three axes
- The Distal Interphalangeal (DIP) joint of the middle finger (Middle03), mostly around the Y-axis
- The Proximal Interphalangeal (PIP) and DIP joints of the index finger (Index02 and Index03), again around the Y and Z axes

We initially tried to manually clamp the rotation values, based on the finger closure range. For instance, if the closure of the index finger was between two chosen thresholds, we would clip or adjust the output rotation of Index03 around the problematic axis.

Although it partially worked, this method failed for the thumb, where closure and abduction moved simultaneously. It was very difficult to identify a clean interval to apply corrections. Even when thresholds were effective, the resulting motion appeared unnatural in the post-processed intervals.

Sine/Cosine Encoding: To address the discontinuity and ambiguity problems caused by the Euler angle wrapping (e.g. $+180^\circ = -180^\circ$), we introduced sine and cosine encoding for problematic joint angles. Instead of predicting

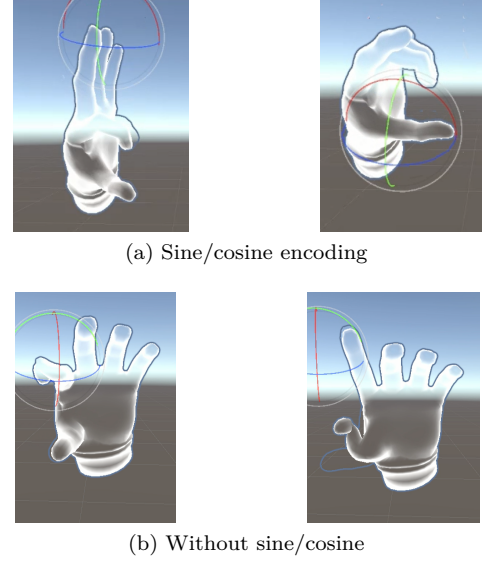


Fig. 2: Finger pose prediction using different strategies.

a raw angle θ , the model predicts $\sin(\theta)$ and $\cos(\theta)$, which ensures continuity (no sharp jumps at $\pm 180^\circ$) and geometric uniqueness.

The decoding was performed using: $\theta = \arctan2(\sin(\theta), \cos(\theta))$

This technique was applied selectively to:

- All thumb joints
- Middle03
- Index02 and Index03

With this fix, the motion of the fingers became visually stable and biomechanically accurate.

Z-Score Filtering for Outlier Removal: We also applied z-score filtering to eliminate outliers from the dataset before training. The z-score of a value is defined as: $z = \frac{x - \mu}{\sigma}$ where:

- x is the sample value
- μ is the mean across the dataset
- σ is the standard deviation

We computed the z-score for each joint angle component, and excluded any sample where any value exceeded a z-score of ± 2.5 . This threshold was chosen because:

- In a normal distribution, 98.8% of values fall within ± 2.5 standard deviations
- This filters out only the most extreme 1.2% of data, likely to be noisy samples or sensor errors

This step helped clean the dataset while preserving the valid data, ensuring that our models were not influenced by noisy samples.

Standardization: Joint angle values were standardized using a StandardScaler: after the application of sine and cosine encoding, all joint angle values were centered around the mean and scaled to unit variance. This preprocessing step ensures numerical stability and promotes faster convergence during model training.

B. Dimensionality Reduction

Building on the initial approach of using regression models to map the four input parameters to hand joint angles expressed via sine and cosine transformations, our focus shifted toward exploring how dimensionality reduction techniques could enhance both reduce complexity of the task and biomechanical realism in motion prediction.

We explored multiple strategies to incorporate postural synergies into the learning process:

1) Direct PCA Output

In an initial approach, we trained both the FCNN and Transformer models to directly predict the principal components (PCA coefficients) of the preprocessed joint angle data. The models were trained to minimize the MSE loss between the predicted and ground-truth PCA components.

2) PCA in the Loss

Also, the models were trained to predict the full dimensional representation of joint angles, but the loss was computed in the PCA space. Specifically, the MSE was applied between the PCA projection of the model's output and that of the ground truth. This formulation allowed us to retain the expressiveness of the original output space while encouraging the network to focus on capturing the most significant motion patterns.

3) Latent-Space Constraints from Autoencoders

To explore nonlinear dimensionality reduction, we trained an autoencoder to learn a compact latent representation of the joint angle space. The FCNN and Transformer models were then trained to predict the full output, while using a loss computed in the autoencoder's latent space, comparing the encoded output of the prediction to that of the ground truth. This approach allows for richer, learned synergies beyond linear PCA while maintaining compatibility with the existing regression models.

C. Models

Fully Connected Neural Network: The architecture of this model consists of a series of four Linear Layers with the following dimensions:

Input (4) → 512 → 256 → 128 → 64 → Output (62)

Between each Linear Layer, there is a **LeakyReLU** activation function, followed by either a **BatchNorm** or a **Dropout** layer.

This model features a simple architecture, enabling very short training times. It is well suited for analyzing direct and statistical relationships between input and output. Due to its low computational demands, it achieves faster inference speeds.

However, it has limited generalization capabilities for temporal dynamics or sequential data, highly non-linear relationships, and noisy input conditions.

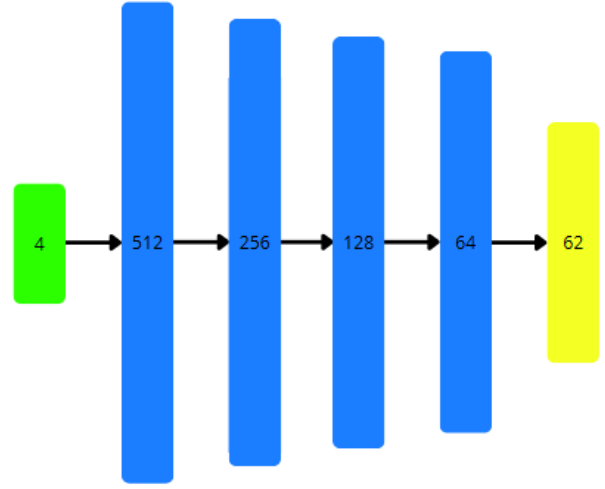


Fig. 3: FCNN Architecture

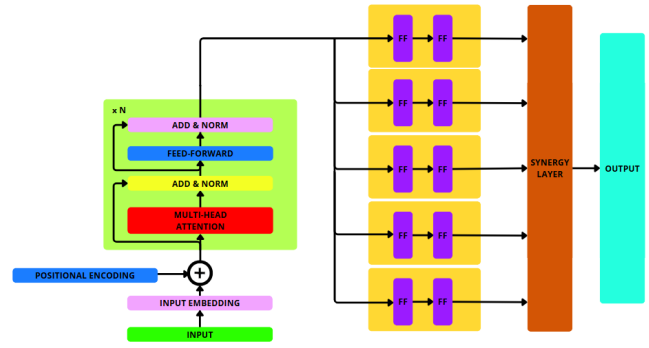


Fig. 4: Transformer Architecture

Transformer-Based Neural Network: Architecture Flow

Input → (Input Embedding) → (Positional Encoding) → (Encoder) → (Output Heads) → (Synergy Layer) → Output

Key components:

- **Input Embedding** [Linear → GELU → Linear → LayerNorm] Adapts the input dimension to the expected transformer size.
- **Positional Encoding** Computed using sine and cosine functions, then added to the input. Helps the model capture temporal dependencies between poses if the dataset is composed by an ordered sequence of poses.
- **Transformer Encoder** A stack of 3 Encoder Layers, each containing:
 - Multi-Head Attention (MHA) Layer (128-dim, 4 heads of size 32)
 - Add & Norm (Residual connection + Layer Normalization)
 - Feed-Forward (FF) Layer (512-dim)
 - Add & Norm

- **Output Heads** A dictionary of 5 heads (one per finger). Each head consists of: [Linear $\rightarrow$ GELU $\rightarrow$ Linear]. Each head predicts a dynamic number of outputs, depending on how many joint angles are split into sine/cosine components.
- **Synergy Layer (Optional)** [Linear] Activated only when the network has to predict finger synergies instead of joint angles.

Proposal Design: The proposed design for this architecture is based on two parts: the first part consists of the transformer encoder, designed to analyze the correlations between finger movements; while the second part is composed of a dictionary with 5 output heads, each responsible for predicting the final joint values for each individual finger. In addition, there is an optional layer, a simple linear layer that takes as input the concatenation of the outputs heads and returns their synergies.

It should be noted that when the optional layer is not used, the outputs of the heads exactly match the joint values to be predicted. However, if the synergy layer is used, then the outputs of the heads will no longer represent the joint values of the individual fingers, since these will instead be part of the network as components for predicting the synergies. Therefore, the values generated by the heads will relate to the synergies rather than to the joints.

Advantages & Trade-offs: Captures complex nonlinear relationships between closure parameters and joint rotations. Requires a larger dataset for effective training and has a longer training time. Despite higher computational cost during inference, it remains real-time capable (as demonstrated in our use case).

Feature	FCNN	Transformer
Architecture	Simple (Dense Layers)	Complex (Self-Attention + Heads)
Training Speed	Very Fast	Slower (Needs More Data)
Inference Speed	Extremely Fast	Fast (Real-Time Viable)
Generalization Strength	Limited (Linear / Simple Cases)	Strong (Nonlinear / Noisy Data)

TABLE I: Comparison of architectural, training, and generalization characteristics between the FCNN and Transformer models.

D. Unity

To simulate and visualize real-time human hand motion, the project was developed in the Unity game engine, where a 3D hand model was rendered and animated. The Weart Unity Software Development Kit (SDK) was integrated into the environment, and the Weart Left Hand prefab was used to represent the hand’s anatomical structure. Each finger consists of three joints: proximal (01), intermediate (02), and distal (03) phalanges, resulting in 15 joints in

total, each with three rotational degrees of freedom (DoF) along the x, y, and z axes. Joint rotations are computed and applied relative to the palm’s reference frame to ensure anatomically plausible and consistent articulation and then to maintain consistency regardless of global orientation.

Dataset Generation : To construct the dataset for model training, a custom C# script (HandDataLogger.cs) was developed within Unity. This logger captures data at **20 Hz**, recording both the rotational states of the 3D hand model and the corresponding sensor inputs from the Weart haptic glove which are real-time parameters defined in the WeArtThimbleTrackingObject components attached to the thumb, index, and middle fingers.

Real-Time Prediction Framework : Following the training phase, a real-time inference pipeline was implemented using a client-server architecture to connect Unity with the Python-based neural network.

Unity acts as the client, acquiring four input values from the Weart TouchDIVER device: the input parameters of the thumb, index and middle fingers, along with the abduction parameter of the thumb. These float32 values are serialized into a byte stream and transmitted via a TCP socket to the Python server. The Python server, operating independently, receives the incoming data, reshapes it into a tensor, and feeds it to a pre-trained neural model. The model output then is converted, when needed, into 45 values, corresponding to the predicted Euler angles for the 15 joints. The predicted joint rotations are then sent back to Unity via the same TCP connection. Unity deserializes the received data and applies the joint rotations to the simulated hand using the Weart SDK Controller in Unity.

IV. EXPERIMENTS

A. Direct PCA Output

In this section, we analyze the initial experiments carried out during the early stages of the project. The objective was to train various models capable of predicting the principal components obtained through PCA. To this end, models were trained to output different numbers of principal components.

After experimentally identifying a list of **17 joint angles** related to the finger phalanges to be fixed, a **reference model** was defined. This model does not directly predict the 45 joint angles but instead outputs a vector of **62 values**, including angles, sines, and cosines. This configuration serves as the baseline for comparison with the other model variants; we will refer to this model as T-62 or F-62 (Transformer or FCNN) and to the model that directly predicts 45 joint angles as T-joints or F-joints.

The alternative models were designed to predict **45, 30, 15, and 10 PCA components** and we will refer to them respectively as T-45/F-45, T-30/F-30, T-15/F-15, T-10/F-10. The decision to apply PCA starting from the reference model is motivated by the fact that this configuration demonstrated the best overall performance. Therefore, it

is particularly relevant to analyze how the use of PCA influences prediction quality.

Since the **hand model requires exactly 45 joint angles** as input, the output of the neural networks must be reconverted into the appropriate format. Specifically:

- The reference model performs a *conversion from sine and cosine to angle* for the fixed joints;
- The PCA-based models perform two sequential transformations: first, a *PCA inverse transformation* to reconstruct the 62 values, followed by a *final conversion to the 45 joint angles* required by the hand model.

B. PCA-Based Loss Only

The project also explores a novel loss formulation for regressing the full 62-dimensional hand pose from four high-level closure parameters. Rather than directly minimizing the Mean Squared Error (MSE) between the predicted and ground truth joint representations, we leverage PCA to compute the loss in a lower-dimensional latent space. This approach should encourage the models to focus on the most important motions in the joint data.

The training process is implemented in the script `train_losspca.py`. It supports two model architectures (FCNN and Transformer) which are detailed in the previous section. In this experiment, we do not train the models to predict the PCA components directly. Instead, we predict the full 62-dimensional joint representation, but compute the **loss in the PCA space**. Specifically:

- A PCA model is first fit to the full output space, capturing almost all of its variance.
- During training, the predictions and ground truth outputs are projected into this PCA space, and the loss is computed as the MSE between the projections.
- This should encourage the model to learn to match the most relevant subspace of the joint pose distribution, while still outputting interpretable joint values.

C. Autoencoder-Based Augmented Loss

In a further development of the PCA-based loss approach, we replaced the linear dimensionality reduction of PCA with a nonlinear autoencoder. Unlike PCA, the autoencoder is capable of learning a latent space that better captures complex non-linear synergies between finger joints. This latent space is not used as the output target of the regression models but only to define the loss.

We use a standard symmetric feed-forward autoencoder over the 62-D joint representation: encoder $62 \rightarrow 512 \rightarrow 256 \rightarrow L$ and decoder $L \rightarrow 256 \rightarrow 512 \rightarrow 62$, with ReLU activations in the hidden layers and linear latent/output layers (see Fig. 5). We train the autoencoder on the entire dataset (joint representations only), sweeping the latent dimensionality $L \in \{10, 15, 30, 45\}$. After training, the encoder f_{enc} is *frozen* and used solely to compute the latent loss for the FCNN and Transformer predictors; the decoder is not used during predictor training.

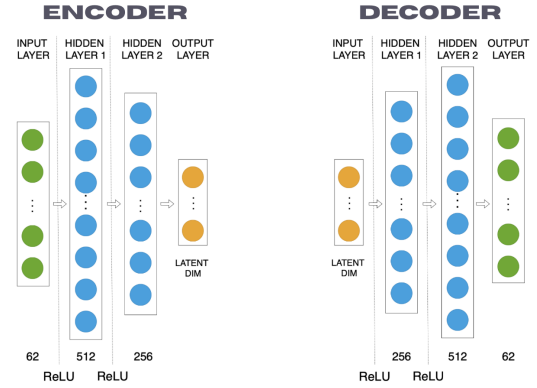


Fig. 5: Autoencoder Structure

In particular, we trained the FCNN and Transformer models to predict the full 62-dimensional joint representation, as in the baseline case. However, during training, the output prediction and the ground truth are both passed through the pretrained encoder:

$$\mathcal{L}_{\text{latent}} = \|f_{\text{enc}}(\hat{y}) - f_{\text{enc}}(y)\|^2 \quad (1)$$

Basically, we took both the ground truth output and the model's prediction (each a 62D vector), passed them through the encoder, and then computed the MSE between their latent representations. This way, we pushed the prediction to lie close to the correct pose, in terms of how that pose "behaves" in the compressed space of plausible hand movements.

This method encourages the model to generate more natural and consistent hand poses without having to learn or predict the latent vector directly. It's also more efficient: for example, when the latent space is just 10 dimensions, computing the loss is much lighter than in full 62D space. Training is faster, and the model still learns to output realistic results.

During training, the encoder weights are kept frozen, and only the regression model is optimized. The predicted outputs consist of 62 scalar values, representing the sin/cos-encoded joint angles. This format is directly compatible with the real-time Unity pipeline.

Constraint Loss Term: In the initial experiments, the prediction models were trained using only a latent-space loss, where the predicted and ground-truth joint configurations were passed through a pre-trained autoencoder and compared in the compressed latent space. While this approach worked well in terms of global hand pose consistency, we noticed that the model could still produce implausible joint configurations, especially for individual joints with more freedom or weaker correlation to others.

To address this, we added a constraint loss term that penalizes joint predictions that fall outside the observed range in the dataset. For each joint component, we computed the empirical minimum and maximum values from the training set. The constraint loss is defined as:

$$\mathcal{L}_{\text{constraint}} = \frac{1}{n} \sum_{i=1}^n \left[\text{ReLU} \left(y_{\min}^{(i)} - \hat{y}^{(i)} \right)^2 + \text{ReLU} \left(\hat{y}^{(i)} - y_{\max}^{(i)} \right)^2 \right] \quad (2)$$

where:

- $\hat{y}^{(i)}$ is the predicted value for joint i
- $y_{\min}^{(i)}$ and $y_{\max}^{(i)}$ are the minimum and maximum values for that joint in the dataset
- ReLU is the Rectified Linear Unit, which ensures the penalty is only applied when predictions exceed the limits

This loss was added to the total loss during training:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{latent}} + \mathcal{L}_{\text{constraint}} \quad (3)$$

After introducing this constraint loss, we observed a clear improvement in model behavior. The outputs were more stable, joints stayed within realistic limits, and the overall training process became more reliable. Validation loss consistently improved, and the poses looked more natural during real-time inference in Unity.

V. RESULTS

In this section, we present the main results obtained by comparing the models in the Unity simulator.

A. Direct PCA Output

FCNN: Initially we compared F-62 and F-joints models observing the different behaviors, illustrated previously in the Sine/Cosine Encoding section: F-62 model can better represent the hand posture with respect to the F-joint model (for this comparison only graphical results are reported).

Next, we compare the prediction results of the other models to understand how PCA dimensionality influences hand posture prediction. When using PCA to reduce the complexity of our problem, we observed that there is no significant drop in performance between 45, 30 and 15 PCA components, but with the model F-10 the index finger can't be reconstructed correctly. This suggests that we can highly reduce the complexity of our problem, but at the same time still get some error in reconstruction.

Transformer: For the Transformer architecture, we analyzed the same differences in the T-62 and T-joint models, highlighting the importance of Sine/Cosine Encoding (also here there is a graphical analysis).

As shown in the image 2 in the Sine/Cosine Encoding section, the T-joints model struggles to accurately predict the hand posture in certain configurations, resulting in performance drops. In contrast the T-62 model handles these scenarios more effectively, maintaining reliable prediction quality.

Next, we compare the prediction results of the other models. When using PCA to reduce the complexity of our

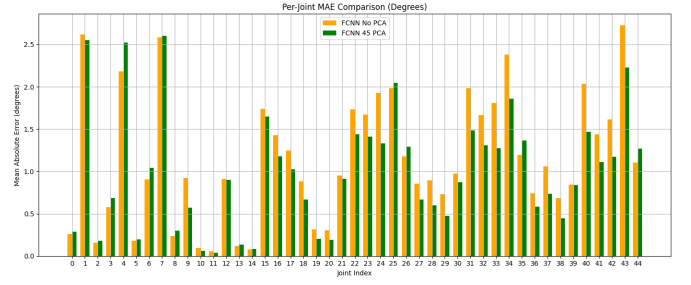


Fig. 6: Average degree error for each angle, FCNN with PCA 45 and FCNN 62.

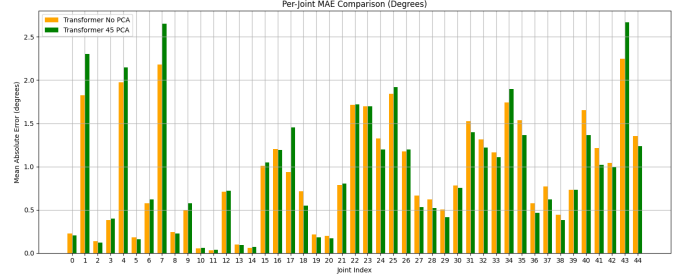


Fig. 7: Average degree error for each angle, Transformer with PCA 45 and Transformer 62

problem, we observed that there is no significant performance drop, even when using only 10 PCA components, which account for 98% of the total hand posture variance represented in the dataset. This suggests that we can substantially reduce the complexity of our problem without losing important information.

In Figs. 6 and 7 it is possible to analyze the per-joint MAE (in degrees) across all 45 joints between F-45 and F-62 models, and between T-45 and T-62 models. Trying to predict synergies, both FCNN and Transformer models get the same error pattern, which justifies the similar behavior at inference time. As you can notice, if the task is simplified (e.g. the task to solve is to predict 45 elements instead of 62) the FCNN seems to behave better while for the Transformer there are no big variations in performances. But if the problem is oversimplified, both models start to lose performance, as seen in Figs. 8 and 9. Our baseline models achieve a per-joint MAE of approximately 2.5 degrees. Compared to related work such as that of Glauser et al. [7], which reported a mean hand pose error of around 15 degrees, our approach demonstrates a significant result in accuracy. This solid foundation makes our framework a reliable and practical solution for real-time applications and serves as a testament to the power of our data acquisition and preprocessing pipeline.

B. PCA-Based Loss

To evaluate the trained models in a real-time scenario, we developed a separate script to run the prediction server and stream the joint angles into a virtual environment

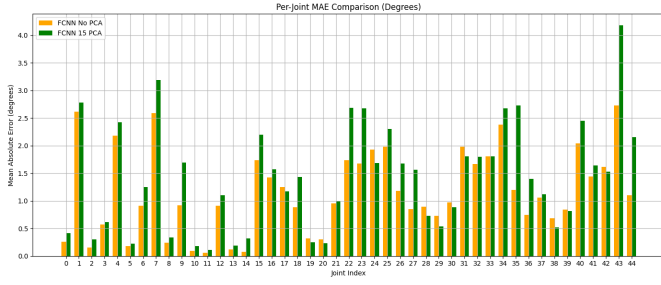


Fig. 8: Average degree error for each angle, FCNN with PCA 15 and FCNN 62.

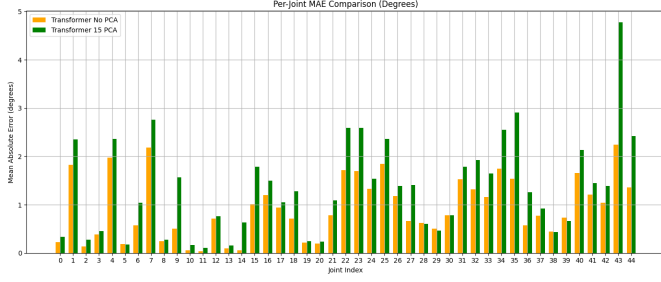


Fig. 9: Average degree error for each angle, Transformer with PCA 15 and Transformer 62

N PCA Components	Best Test MSE Loss	
	FCNN	Transformer
45	0.0070	0.0059
30	0.0104	0.0089
15	0.0190	0.0134
10	0.0190	0.0158

TABLE II: Training results with PCA in the loss domain.

such as Unity. While the test loss remains consistently low across all training runs, as reported in Table III for both the FCNN and Transformer models, visual inspection reveals significant issues with joint-level accuracy.

Notably, the overall hand behavior (such as opening and closing of the hand and abduction of thumb) is captured well and feels responsive in the virtual environment. However, individual joints exhibit unnatural or erratic rotations, particularly at intermediate or fine levels of articulation. This issue becomes clearly observable in the demonstration videos provided in the `video_results/training_losspca_results` folder.

To better understand the impact of dimensionality reduction, we conducted experiments using 45, 30, 15, and 10 PCA components. Although the PCA reconstruction loss remains low, we observed that joint-level prediction quality deteriorates significantly when fewer than 45 components are used. This behavior is illustrated in Figs. 10 and 11, which compares the average absolute prediction error (in degrees) for each joint between the models trained with 45 and 15 PCA components. The 15-component model introduces substantial errors in many joints, including

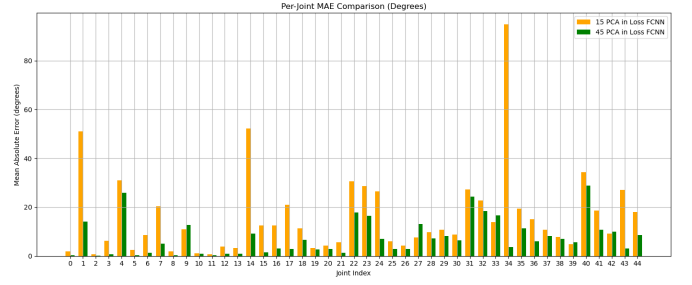


Fig. 10: Average degree error for each angle with PCA in the loss of the FCNN model (45 components, 15 components)

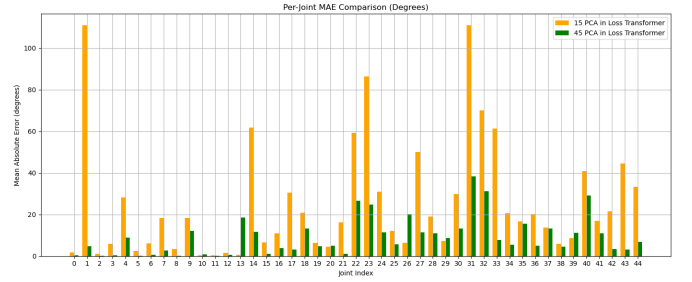


Fig. 11: Average degree error for each angle with PCA in the loss of the Transformer model (45 components, 15 components).

dominant ones, despite having similar PCA loss.

When computing MSE in the PCA domain, there is a compression of the output space and the model only learns to minimize error in this compressed space, not directly in the joint angle space. As a result the PCA reconstructed output may look good statistically, but small errors in PCA space can cause big deviations in original joint angles after inverse PCA. So the model is not penalized for inaccurate predictions of specific joints, especially the non-dominant ones.

C. Autoencoder-Based Augmented Loss

This section presents the results obtained from training and evaluating the FCNN and the Transformer model using a constrained latent-space loss.

Performance is evaluated both **quantitatively**, using the MAE computed per joint over the entire test set, and **qualitatively**, through visual inspection of the predicted hand movements in Unity. Additional experiments explore the impact of the latent space dimensionality and the effect of constraint-based regularization on the stability of the model.

In particular, Figs. 12 and 13 compare the per-joint MAE (in degrees) across all 45 joints, for both the FCNN and the Transformer-based models. Fig. 12 shows the results using a latent space of 15 dimensions, while Fig. 13 uses a latent space of 45 dimensions.

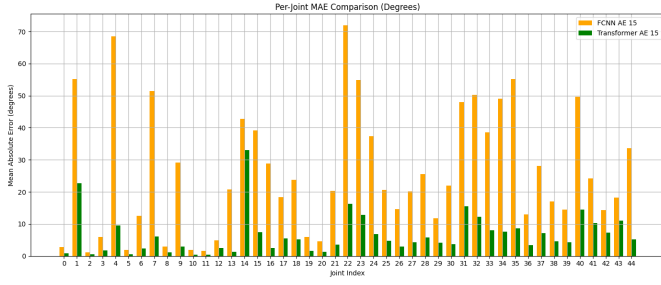


Fig. 12: Average degree error for each angle with Autoencoder Based-Loss, latent space 15 (Transformer, FCNN)

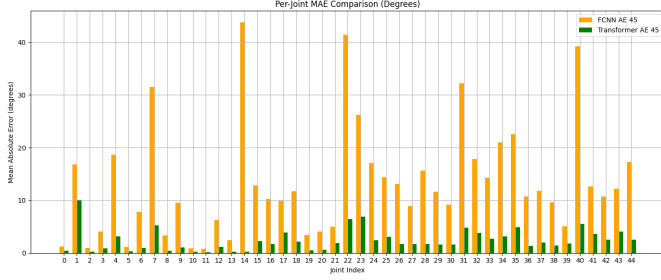


Fig. 13: Average degree error for each angle with Autoencoder Based-Loss, latent space of 45 (Transformer, FCNN)

We can observe that the Transformer consistently achieves lower MAE values across the majority of joints, especially in those with higher variability or greater freedom of movement. This trend is even more pronounced in the case of the 45-dimensional latent space (Fig. 13), where the performance gap between the FCNN and the Transformer becomes more significant. These results suggest that the Transformer model is better at capturing complex joint dependencies and benefits more from richer latent representations. On the other hand, when the latent space is smaller (15 dimensions), both models see a performance drop, but the Transformer still maintains a noticeable advantage. An important finding is that the visual simulations revealed that using the Transformer trained with a 15-dimensional latent space autoencoder strikes a good balance between performance and efficiency. It delivers visually consistent and natural hand motion while significantly reducing training complexity and computational cost, making it a lightweight yet effective solution for real-time applications.

Moreover, this observation is supported by the results reported in Table III, where the best test MSE loss achieved by the Transformer is consistently lower than that of the FCNN across all tested latent space dimensions.

VI. CONCLUSION

We presented a data-driven framework for motion retargeting based on minimal sensing, using only four high-level parameters, which can be easily measured with simple sensors, to reconstruct full hand configurations. Among the tested models, the Transformer combined with an

Latent Space Dimension	Best Test MSE Loss	
	FCNN	Transformer
10	0.0062	0.0053
15	0.0061	0.0051
30	0.0043	0.0036
45	0.0049	0.0040

TABLE III: Training results with Encoder in the Loss.

autoencoder-based loss proved most promising, capturing nonlinear dependencies and synergies while ensuring biomechanically plausible motion. Thanks to the use of closure parameters rather than absolute keypoints, the method is inherently adaptable to hands of different sizes and morphologies, making it a compact and versatile alternative to keypoint-rich or optimization-heavy approaches, with potential for seamless extension from virtual environments to real-world teleoperation.

Looking ahead, several directions could further enhance our framework. First, it could be worth assigning higher weights in the loss function to biomechanically critical joints while allowing minor deviations in less critical ones. This strategy could reduce discontinuities caused by hard thresholds and encourage smoother, anatomically plausible predictions, effectively combining realism with the flexibility of a data-driven model. Also, although the Transformer already models sequences via positional encoding, incorporating explicit time-series data could allow the model to exploit not only the current glove inputs but also longer motion history, potentially improving temporal consistency and predictive accuracy. Related to this, Recurrent Neural Networks or similar sequence-oriented models could be tested to achieve smoother temporal consistency. The Transformer architecture itself could be further enhanced, for instance by increasing depth or capacity to capture more subtle inter-finger dependencies. In addition, extending the output to include kinematic properties such as velocities or accelerations would enrich the motion representation. Finally, integrating force feedback by combining reconstructed poses with the haptic features of the Weart glove could create a more immersive and realistic interaction.

VII. LIMITATIONS

Despite the promising results, our work presents some limitations that should be acknowledged. First, the dataset was collected using a single subject and a single hand morphology, therefore the ability to generalize to different hand sizes and motor patterns remains to be validated. Second, the sensing setup is inherently limited: the Weart TouchDIVER G1 does not provide dedicated sensors for the ring and pinky fingers. Consequently, independent configurations of these fingers cannot be reconstructed, which restricts the expressiveness of the retargeted motions. Third, no explicit collision handling between fingers was

implemented in this version, since our primary focus was on continuous motion reconstruction rather than physical interaction. These aspects open concrete directions for future development, such as multi-user data collection, enriched sensing for all five fingers, and the integration of biomechanical constraints or collision-aware models, in line with the extensions already outlined in the conclusion.

REFERENCES

- [1] “Weart website,” <https://weart.it/>.
- [2] “Weart developer guide td g1,” <https://weart.it/developer-guide-td-g1/>.
- [3] M. F. Marco Santello and J. F. Soechting, “Postural hand synergies for tool use,” *The Journal of Neuroscience*, 1998.
- [4] R. Wen, Z. Ren *et al.*, “Dexterous teleoperation of 20-dof bytedexter hand via human motion retargeting,” 2025.
- [5] Z.-H. Yin, C. Wang, L. Pineda, K. Bodduluri, T. Wu, P. Abbeel, and M. Mukadam, “Geometric retargeting: A principled, ultra-fast neural hand retargeting algorithm,” 2025.
- [6] C. Zimmermann and T. Brox, “Learning to estimate 3d hand pose from single rgb images,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 4913–4921. [Online]. Available: <https://doi.org/10.1109/ICCV.2017.524>
- [7] O. Glauser, D. Wu, D. Panozzo, O. Hilliges, and O. Sorkine-Hornung, “Interactive hand pose estimation using a stretch-sensing soft glove,” *ACM Transactions on Graphics (TOG)*, vol. 38, no. 4, pp. 1–15, 2019. [Online]. Available: <https://doi.org/10.1145/3306346.3322957>
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [9] C. Guo, Y. Mu, X. Zuo, P. Dai, Y. Yan, J. Lu, and L. Cheng, “Generative human motion stylization in latent space,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.13505>
- [10] A. A. Portnova-Fahreva, F. Rizzoglio, F. A. Mussa-Ivaldi, and E. Rombokas, “Autoencoder-based myoelectric controller for prosthetic hands,” *Frontiers in Bioengineering and Biotechnology*, vol. Volume 11 - 2023, 2023. [Online]. Available: <https://www.frontiersin.org/journals/bioengineering-and-biotechnology/articles/10.3389/fbioe.2023.1134135>

All source code used in this work is publicly available at: <https://github.com/serenatrovalusci/Hand-Motion-Reconstruction-from-Minimal-Inputs-through-Latent-Space-Learning.git>

